Regularized trace to marked point action, with P-192 normalization

Explicit graph kernel and quadratic shift
Saved small-field kernel; dense coefficient construction charged

Regularize trace; evaluate at P=-U
N(-U) / (4*y(U)^2*f_r_prime(x(U))^2) = x(R0+psi(U))

Lift and select the rational point
Marked action psi(U)=[x]U; require 2R0 nonrational

Repeat action, then implicit BSGS
d-1 actions produce [x^d]G; charge both search stages

Construction, 2026-11-08. For the mixed graph kernel H={(u,psi(u)):u in E[r]} from rounds53-54, the principal part of the mixed trace determines the point psi(U), rather than only a quotient-curve identity. The target is P-192; the complete action and auxiliary-power controls below use E/F11:y^2=x^3+2x+7, r=7, G=(6,2). No novelty claim is made for the auxiliary-input discrete-log stage. [pcc:unverified]

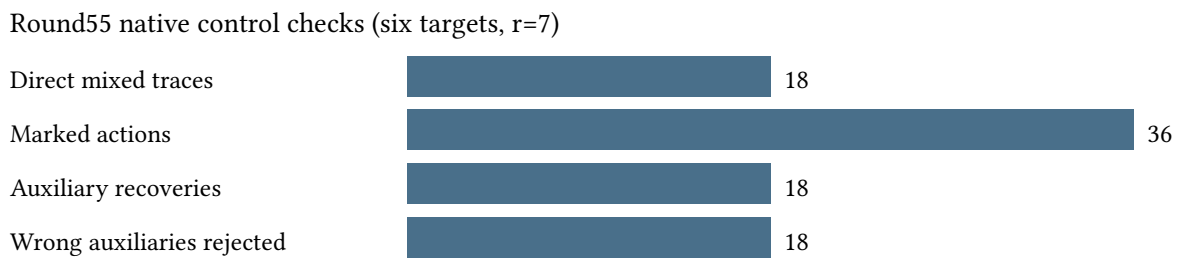
Domain and normalization. Let E:y^2=x^3+ax+b over F_q have characteristic greater than3, and let r be an odd prime different from the characteristic. Write f_r(X) for the standard odd division polynomial, with leading coefficient r and simple roots x(u) for nonzero u in E[r] modulo sign. Choose R0 in E(F_q^2) outside E[r], with 2R0 nonrational. Set C(P,R0)=sum_u x(P+u)x(R0+psi(u)), D=f_r^2, and N(P)=D(x(P))C(P,R0)=A(X)+Y B(X). For a Frobenius-stable kernel, coefficients lie in F_q^2. The numerator is regular at every nonzero r-torsion point. [pcc:unverified]

Principal-part identity. For U=(u,v), the addition formula gives x(P+U)=[uX^2+(a+u^2)X+au+2b-2vY]/(X-u)^2. At P=-U, its numerator is 4v^2. The term belonging to -U has numerator zero there; all other terms have no pole there. Since f_r has a simple root at u, cancellation gives N(-U)=kappa(U)*x(R0+psi(U)), with kappa(U)=4v^2*f_r'(u)^2 nonzero. This is a written algebraic argument; the full function-field and local-regularity theorem is not formalized in Lean. [pcc:unverified]

Explicit point action. Evaluate N(-U)/kappa(U), lift its x-coordinate to the two curve points over F_q^2, and subtract R0. When psi(U) is rational, the candidates are psi(U) and -psi(U)-2R0. Exactly one is rational because 2R0 is nonrational. On the rational cyclic subgroup, psi(U)=[x]U for Q=[x]G. This extraction takes the numerator and public point U; the target logarithm is not an extractor input. The finite control uses R0=6S, with S of order17 and Frobenius(S)=-S, so its sign-selection hypotheses hold. [pcc:unverified]

Auxiliary-input stage. Starting with Q, d-1 successive actions produce [x^d]G. For d dividing r-1, put L=(r-1)/d and choose a certified primitive generator g of F_r*. An implicit baby-step/giant-step on [g^(d*k)]G finds k2=log_g(x) modulo L from [x^d]G. Normalize Q by g^(-k2); an implicit search with generator g^L then finds k1 modulo d. Return x=g^(k2+L*k1), checking both Q=[x]G and the auxiliary point. The searches use O(sqrt(L)+sqrt(d)) scalar multiplications and O(log r*(sqrt(L)+sqrt(d))) ordinary group additions, plus generator certification and setup. This is the established Cheon p-1 auxiliary-input mechanism, with its elementary two-stage derivation spelled out here. [pcc:unverified]

Native exact checks. The Rust checker independently implements the F_(11^3)[W]/(W^2-2) field, curve arithmetic, polynomial evaluation, action extraction and two-stage search. It reconstructs f_7 from all24 torsion x-roots and all six complete numerator coefficient arrays from the saved graph maps; checks A degree49 and B degree46 and coefficient descent; directly recomputes18 finite mixed traces; checks36 principal-part actions across all six nonzero rational targets and all six nonzero rational inputs; and recovers18 scalars for d=2,3,6. It rejects18 incorrect auxiliary points. Fresh-copy replay reproduces the entire native JSON byte for byte. Separate mutations of a numerator coefficient and a P-192 derivative both terminate with assertion rejection, exit101. These are producer-side arithmetic checks, not a trusted signed PCC verdict. [pcc:unverified]



Unit: exact check count. Source: native-verification.json. No timing inference.

Actual P-192 component. Use p=6277101735386680763835789423207666416083908700390324961279 and r=6277101735386680763835789423176059013767194773182842284081, with the standard a=-3 and b from round52/input.json. The native checker evaluates the original division-polynomial recurrence with dual-number x derivatives, including the original even-index factor1/(2Y), independently of the saved parity-normalized recurrence. At G, the public Q fixture, and2G, it reproduces f_r=0, nonzero f_r' and the saved kappa values. Each original recurrence visits1508 memoized indices. The complete decimal points, derivatives and normalizations are in native-verification.json. No P-192 N, marked action or Cheon execution is present. [pcc:unverified]

Cost ledger. Current dense construction stores Theta(r^2) coefficients; the saved simple coefficient builder requires O(r^4) field operations after explicit kernel availability. Charge T_kernel+T_N+(d-1)*(T_evalN+T_normalization+T_lift_and_select)+T_Cheon+T_generator_setup, retaining all point operations and final checks. Dense Horner evaluation costs O(r^2) field operations per action. For P-192, d=16*5*2389=191120 divides r-1; producing its auxiliary point would require191119 actions. The two search widths are ceil(sqrt((r-1)/d))=1812285081351063599140527 42 and ceil(sqrt(d))=438. This divisor is an admissibility check, not an optimized or executed P-192 attack. The concrete next task is a compressed evaluator for N(-U), preserving this principal coefficient without enumerating the kernel or dense numerator. [pcc:unverified]

Provenance and limits. The saved Python producers and their outputs are retained as historical source and certificates; they were not extended or rerun during PR preparation. The supported replay path is native Rust, with locked dependencies and no Python subprocess. The P-192 constructor input contains only public curve parameters and points. The tiny verifier enumerates the six possible scalars to check controls; it does not supply them to action extraction. Counts are exact arithmetic diagnostics; there are no native timing, cloud, statistical or speedup claims. The broad legacy Rust library command and its actual outcome are recorded separately in validation.json. [pcc:unverified]

Sources checked. Earlier rounds53-54 supply the explicit mixed kernels, trace and pole-free shift. Andrew Sutherland, MIT18.783 Lecture5 (2025), slides14-16, supplies the standard division-polynomial recurrences: https://math.mit.edu/classes/18.783/2025/LectureSlides5.pdf . Cheon's author PDF, https://www.math.snu.ac.kr/~jhcheon/publications/2010/StrongDH_JoC_Final2.pdf , timed out; the PKC2012 archive PDF https://www.iacr.org/archive/pkc2012/72930594/72930594.pdf returned403. Therefore the implemented two-stage search is justified by the explicit derivation above and exact controls, not a newly verified reading of those full texts. Sources checked (KB): KB unavailable in this runtime; no research-kb MCP tools are loaded and scripts/research-kb is absent in the migrated checkout. No absence or priority claim follows from this gap. [pcc:unverified]